

VMI Analytics Platform

Epic 1 — Platform Standards & Architecture

Discovery Documentation | Microsoft Fabric Migration Initiative

Sazerac VMI Team | Conner Ruth | May 2026

About This Document

This document captures the finalized architecture decisions, naming standards and development standards established in Epic 1 of the VMI Analytics Fabric migration. It is intended as a living reference for all team members building on the platform. All standards in this document must be followed before source pipelines are built.

Project Epics Overview

Epic 1 (this document) is the first of seven epics governing the VMI Analytics platform build. The remaining epics are:

- **Atlas Inventory & Lineage Reverse Engineering**
- **Core Fabric Infrastructure**
- **Atlas Source Migration**
- **Atlas Master Pipeline**
- **Scheduling + QA**
- **Expansion Roadmap**

Table of Contents

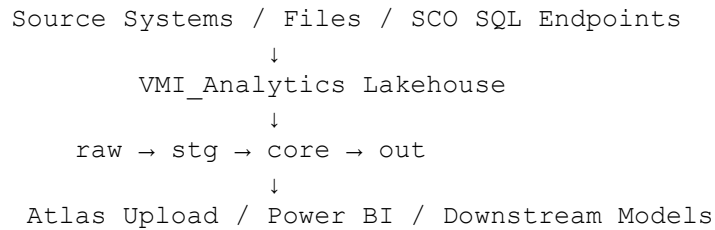
- [1.1 Decide Fabric Architecture](#)
- [1.2 Decide Folder and Zone Structure](#)
- [1.3 Define Object Types by Layer](#)
- [1.4 Define Cross-Workspace Query Strategy](#)
- [1.5 Define Credential Strategy](#)
- [1.6 Set Up Naming Standards](#)
- [1.7 Define Data Standards](#)
- [1.8 Define Notebook & Pipeline Standards](#)

1.1 — Decide Fabric Architecture

Decision

Adopt a Lakehouse-first architecture within the standalone VMI_Analytics Fabric workspace.

Architecture Pattern



Key Decisions

- VMI_Analytics remains fully independent from the cluttered SCO workspace
- Existing enterprise SQL data will be accessed via SQL endpoints and Fabric shortcuts where appropriate
- All VMI-owned transformations, config tables, logs and output tables will live in the VMI Lakehouse
- No Fabric Warehouse in V1
 - Warehouse may be introduced later if reporting scale, performance or governance requirements justify it

Rationale

The VMI team's primary objective is to centralize fragmented Excel, Power Query and SQL-based business logic into a governed, maintainable Microsoft Fabric platform that serves as a single source of truth for downstream analytics, models and operational processes. A Lakehouse-first architecture reduces architectural complexity, governance overhead and maintenance burden for a small analyst-led team while aligning naturally with file-heavy ingestion and transformation workloads.

1.2 — Decide Folder and Zone Structure

Decision

Adopt a domain-based Lakehouse structure with standardized table prefixes.

File Zone Structure

```
Files:
  raw/{domain}/
  archive/{domain}/
```

Domain Examples

- inventory
- orders
- master_data
- reference
- forecast
- logistics

Table Prefixes

- raw_
- stg_
- core_
- out_
- cfg_
- log_
- sbx_ (development / testing only)

Notebook Folders

- /dev
- /prod

Layer Rules

Prefix	Purpose
raw_	Landed source data
stg_	Cleaned / standardized source-level data
core_	Centralized reusable business source of truth
out_	Downstream process outputs
cfg_	Configuration / mappings
log_	Operational monitoring / audit
sbx_	Analyst experimentation only
vw_	Analyst convenience views

Rationale

The same source data will support multiple downstream business processes — Atlas, Bailment, OOS and future use cases. Organizing by business process would create duplication pressure and fragmented ownership. Domain-based organization creates cleaner reuse and reinforces the single source of truth architecture.

Business logic should be centralized once in core_, with process-specific outputs generated downstream.

1.3 — Define Which Object Types Belong in Each Layer

Decision

Define strict object ownership by Lakehouse layer so every table, file, notebook and pipeline has a clear purpose, preventing logic sprawl and inconsistent development patterns.

Layer Standards

Layer / Prefix	Purpose	Allowed Objects	Rules
raw_	Land source data with minimal transformation	Source files, SQL snapshots, API extracts, shortcuts, ingestion notebooks	Preserve source grain; no business logic; append-only where practical

Layer / Prefix	Purpose	Allowed Objects	Rules
stg_	Clean and standardize individual sources	Cleaned Delta tables, validation outputs, transformation notebooks	Rename columns, cast types, dedupe, normalize nulls; no cross-source business logic
core_	Centralized reusable business source of truth	Integrated Delta tables, reusable joins, conformed business datasets	Joins, mappings, business rules and exclusions allowed; logic must be reusable across multiple downstream processes
out_	Final process-specific outputs	Output Delta tables, export logic, delivery formatting notebooks	Exact downstream contract formatting; final filters, names, ordering and file prep
cfg_	Governed configuration and business mappings	Customer mappings, SKU mappings, source registry, parameter tables	Approved business config only; avoid hardcoding config inside notebooks
log_	Operational audit and monitoring	Pipeline run logs, rejected record logs, schema drift logs, freshness logs	Append-only; used for monitoring, debugging and traceability
sbx_	Analyst experimentation	Prototype tables, temporary notebooks, test outputs	Never a production dependency; clean up periodically
vw_	Lightweight reusable query abstraction	Reporting views, analyst convenience views, simplified business query layers	No critical pipeline checkpoint dependencies; avoid deep nested view chains

Object Standards

- Production persisted tables should be stored as Delta tables by default
- Use views (vw_) for lightweight reusable query logic, analyst convenience or reporting simplification
- Use physical Delta tables for critical production pipeline checkpoints (raw_, stg_, core_, out_, cfg_, log_)
- Atlas production outputs should be persisted as physical out_ tables, not views, to support auditability, reproducibility and operational troubleshooting
- External source mappings should land as raw_ / stg_ first, then be promoted into governed cfg_ objects if approved as VMI-controlled configuration

Key Rule *Business logic should be centralized once in core_, with process-specific delivery logic handled in out_, preventing duplicate transformation logic across Atlas, Bailment, OOS and future downstream processes.*

1.4 — Define Cross-Workspace Query Strategy

Decision

Use a mixed cross-workspace strategy. VMI_Analytics may access SCO and other Fabric data through SQL endpoints or OneLake shortcuts, but production-critical VMI logic should land data into VMI-owned Lakehouse tables before downstream transformation.

Standards

- Use shortcuts for stable shared Fabric tables that should appear local without copying

- Use SQL endpoint queries for targeted pulls, filtering, validation and discovery
- Use VMI-owned raw_ / stg_ tables for production-critical dependencies, especially Atlas inputs
- Promote validated external mappings into governed cfg_ objects when used as official VMI configuration
- Avoid building critical production pipelines directly on long chains of live cross-workspace dependencies

Decision Rules

Scenario	Use
Stable SCO table used by many VMI processes	Shortcut
Need only a filtered SQL extract	SQL endpoint query
Need exploratory validation	SQL endpoint query
Source feeds Atlas production output	VMI-owned raw_ / stg_ table
Source is an official mapping table but VMI needs stable logic	Land to raw_, validate, promote to cfg_
Source is small reference data	VMI-owned cfg_ table
Source changes frequently or schema may drift	VMI-owned snapshot with schema validation

Key Rule
Treat shared enterprise data as upstream sources, not as permanent live dependencies for production-critical VMI processes.

1.5 — Define Credential Strategy

Decision

Adopt a phased credential strategy that prioritizes speed for MVP development while allowing secure automation in the future via service account.

Current State (MVP — Manual Execution)

- Development and manual pipeline execution use analyst credentials
- Analysts may authenticate to SQL endpoints, Fabric objects and approved source locations using existing access
- This allows rapid development without waiting on IT or security dependencies
- Manual execution is consistent with the current state of the Atlas process and acceptable for V1

Future State (Production Automation)

- Scheduled automated pipeline execution should transition to a shared service account or managed platform identity
- A shared service account would allow all three analysts to trigger and automate pipelines without personal credential dependencies
- Production automation should not permanently depend on a single analyst's personal credentials — a password rotation, MFA re-prompt or license change would silently break a scheduled run
- The service account conversation should be started now so it is ready when automation is enabled — this is a V2 requirement with a hard target sprint to be assigned

Credential Standards

- Personal credentials acceptable for development and manual execution
- No hardcoded usernames, passwords or API keys inside notebooks or scripts
- External source access (SQL, SFTP, APIs, shared file locations) should be abstracted and configured cleanly for future automation transition
- Architecture should avoid assumptions that permanently tie execution to a specific individual account

Key Rule *The immediate priority is rapid platform development and Atlas process migration, not enterprise identity engineering. MVP execution can be manual under analyst credentials, while the architecture remains compatible with future automated production execution.*

1.6 — Set Up Naming Standards

Decision

Adopt simple, analyst-readable naming standards across Fabric objects, tables, columns and exported files. Naming should prioritize clarity, consistency and maintainability.

Fabric Object Naming

Object Type	Standard	Example
Workspace	{team}_{function}	VMI_Analytics
Lakehouse	{workspace}_Lakehouse	VMI_Analytics_Lakehouse
Warehouse	Not used in V1	N/A
Notebook	nbook_{layer}_{domain}_{source}	nbook_stg_inventory_johnson_brothers
Pipeline	pipe_{process}_{purpose}	pipe_atlas_daily_upload
Dataflow	dflow_{domain}_{source}	dflow_inventory_breakthru
Shortcut	scut_{source_workspace}_{object}	scut_sco_oracle_buylist

Table Naming Standards

Use prefixes as pseudo-schemas inside the Lakehouse.

Prefix	Purpose	Example
raw_	Landed source data	raw_inventory_johnson_brothers
stg_	Cleaned and standardized source data	stg_inventory_johnson_brothers
core_	Reusable business source of truth	core_inventory_position
out_	Process-specific output	out_atlas_upload
cfg_	Configuration and mappings	cfg_sku_mapping
log_	Audit and monitoring	log_pipeline_runs
sbx_	Sandbox / testing	sbx_inventory_join_test
vw_	Views	vw_inventory_summary

Column Naming Standards

Use lowercase snake_case for all governed table columns.

- No spaces
- No special characters
- No CamelCase
- Use consistent business terms
- Abbreviations only when approved

Bad	Good
Customer Name	customer_name
CATN32 Code	catn32
On Hand Qty	on_hand_qty
Inventory Date	inventory_date
Source File	src_file_name
Load Date	load_date

Approved Abbreviations

Term	Standard
quantity	qty
distributor	dist
product	prod
source	src
out of stock	oos
count	count
inventory	inventory
customer	customer
category	category
buy list	buylist

Note: Avoid abbreviating customer, category and buylist due to ambiguity and readability concerns.

Export File Naming

Use this standard:

```
{process}_{output_name}_{yyyymmdd}.{extension}
```

Examples:

```
atlas_inventory_upload_20260512.csv  
bailment_inventory_upload_20260512.csv  
oos_inventory_feed_20260512.csv
```

For reruns:

Key Rule *Names should make the object's layer, domain and purpose obvious without needing to open it.*

1.7 — Define Data Standards

Decision

Adopt consistent data handling rules by layer so pipelines are auditable, repeatable and easier for analysts to maintain.

Null Handling Policy

Layer	Standard
raw_	Preserve source nulls as-is; do not impute or replace values
stg_	Standardize obvious blanks/nulls; flag unexpected nulls in required fields
core_	Apply approved business rules for null handling
out_	No nulls in required output fields unless accepted by downstream process
cfg_	Required fields should not be null
log_	Nulls allowed only where contextually valid

Type Casting Standards

- Explicitly cast all governed columns
- Do not rely on automatic type inference for production tables
- Use consistent types for common fields:
 - dates → date
 - timestamps → timestamp
 - IDs/codes → string
 - quantities → numeric / decimal
 - flags → boolean where practical

Key Rule *Product codes, customer codes, buylist codes and ZIP-like fields should always be stored as strings, not numbers.*

Timestamp / Audit Column Standards

Column	Definition
load_timestamp	When data was loaded into the Lakehouse
load_date	Date partition / processing date
src_file_name	Original file name where applicable
pipeline_run_id	Unique run identifier tying data to log entries

Date Partition Standards

Partition large append-only tables by load_date. Use date partitions for:

- raw_

- stg_
- log_
- Recurring source snapshots

Small config tables do not need partitioning.

Append vs Overwrite Rules

Table Type	Standard
Raw source snapshots	Append
Staged recurring source tables	Append unless rebuilding corrected history
Core history-bearing tables	Append / merge based on business grain
Output production runs	Append by run / date
Config tables	Overwrite or controlled upsert
Log tables	Append only
Sandbox tables	Flexible

Key Rule *Append history-bearing data; overwrite only controlled reference/config objects or explicitly rebuilt datasets.*

Deduplication Policy

Every deduplication step must explicitly define:

- Business key
- Sort / order logic
- Record kept
- Record rejected / logged
- Reason for deduplication

Pattern	Example
Bad	drop_duplicates()
Good	Dedupe by customer_id + catn32 + inventory_date, keeping latest load_timestamp; log rejected records with reason code

Business Key Documentation Standard

Every governed table must document:

Field	Requirement
Table name	Required
Business grain	Required
Business key	Required
Source system	Required
Refresh cadence	Required
Owner	Required
Known exceptions	Optional but encouraged

Example:

```
Table:          core_inventory_position
Grain:          one row per customer + catn32 + inventory_date + source
Business Key:   customer_id, catn32, inventory_date, src_system
```

Source Freshness Expectations

Every recurring source must define:

- Expected refresh frequency
- Latest acceptable file date / load date
- Warning threshold
- Failure threshold
- Owner to contact

Example:

```
Breakthru 852 expected daily before 10 AM.
Warn if older than 24 hours.
Fail Atlas upload if older than 48 hours.
```

Schema Evolution Handling

Change Type	Action
Missing required column	Fail pipeline
New unexpected column	Warn and log
Data type change	Fail or quarantine depending on severity
Column rename	Fail until mapping is updated
Optional column missing	Warn and continue
Required key null spike	Fail or flag for review

Key Rule *Schema changes should be detected and logged, not silently absorbed.*

1.8 — Define Notebook & Pipeline Standards

Decision

Use modular notebooks for transformation logic and Fabric pipelines for orchestration. Notebook code should be standardized, readable, config-driven and validation-heavy so multiple analysts can build and maintain pipelines consistently.

Tool Responsibilities

Object	Role	Standard
Notebook	Performs the work	Extract, validate, transform, write, log
Pipeline	Manages the workflow	Schedule, sequence notebooks, handle dependencies, alert failures
Table	Stores results	Raw, staged, core, output, config, logs

Object	Role	Standard
View	Simplifies access	Lightweight reusable query layer

Notebook Layer Pattern

Use notebook boundaries at logical ownership boundaries.

Notebook Type	Flow	Example
Source notebook	source → raw_ → stg_	nbook_stg_johnson_inventory
Integration notebook	stg_ → core_	nbook_core_inventory_position
Output notebook	core_ → out_	nbook_out_atlas_upload

A single source notebook should generally:

- Read the source data
- Validate raw structure
- Write the raw_ table
- Clean and standardize the data
- Validate the standardized result
- Write the stg_ table
- Log success or failure

Do not split every minor phase into separate notebooks unless the notebook becomes too large or crosses a major logic boundary.

Pipeline Pattern

Use one orchestration pipeline per major business process. Example:

```
pipe_atlas_daily_upload
├── nbook_stg_johnson_inventory
├── nbook_stg_breakthru_inventory
├── nbook_stg_oracle_buylist
├── nbook_core_inventory_position
└── nbook_out_atlas_upload
```

Key Rule *Notebooks contain the logic. Pipelines control the execution.*

Required Notebook Structure

Each production notebook must follow this order:

Step	Section
1	Header / purpose — include notebook name, author, created date and last modified date
2	Imports
3	Config block
4	Helper functions
5	Extract source data
6	Validate raw input
7	Write raw_ table

Step	Section
8	Transform / standardize
9	Validate staged / core / output result
10	Write stg_ / core_ / out_ table
11	Log run status
12	Exception handling / final status

Note: Each notebook should include a header comment block at the top recording the notebook name, author, creation date and last modified date. This provides lightweight version context without requiring full Git integration during early platform development.

Notebook Template

A standard Fabric notebook template should be created and used as the starting point for every new source, integration and output notebook. The template should pre-populate the 12-step structure, config block skeleton, standard function stubs and the header comment block. This prevents deviation from standards and eliminates blank-slate development.

Python Development Standards

Notebook code must be:

- Modular
- Readable
- Commented where business logic is not obvious
- Config-driven
- Consistent across sources
- Free of hardcoded business mappings
- Structured around reusable functions

Preferred Function Patterns

Source notebook:

```
extract_source()
validate_raw()
write_raw()
transform_source()
validate_stg()
write_stg()
log_success()
log_failure()
```

Integration notebook:

```
read_staged_inputs()
validate_inputs()
apply_business_logic()
validate_core()
write_core()
log_success()
log_failure()
```

Output notebook:

```

read_core_inputs()
validate_inputs()
format_output()
validate_output()
write_out()
export_file()
log_success()
log_failure()

```

Config-Driven Pattern

Each notebook should begin with a clear configuration section containing:

- Source name
- Domain
- Source path or source query
- Raw table name
- Staged / core / output table name
- Expected columns
- Required columns
- Business key
- Freshness threshold
- Load date
- File format
- Source owner if known

This keeps source-specific settings in one place and prevents hardcoded values from being scattered throughout the notebook.

Validation & Error Handling Checklist

Step	Validation / Error Handling Needed
Before extract	Config exists; source path/query exists; required parameters populated
Extract	Source file/table reachable; read succeeds; row count > 0
Raw validation	Expected columns present; required source fields present; source freshness valid; row count within expected range
Raw write	Raw_ table write succeeds; rows written match rows extracted; load metadata added
Transform	Required renames applied; data types cast explicitly; null handling rules applied; business key fields created
Staged/core/output validation	Required fields not null; duplicate business key check; valid date ranges; numeric fields valid; mapping success rate acceptable
Rejected records	Invalid/dropped records logged with reason code; no silent row drops
Final write	Target table write succeeds; final row count logged; schema matches expected output
Pipeline logging	Success/failure status written to log_pipeline_runs; rejected rows written to log_rejected_records
Exception handling	Failures logged, then raised so Fabric shows the notebook/pipeline as failed

Assert / Validation Standards

Use assertions or validation functions for:

- Source exists
- Source freshness
- Expected columns
- Required fields not null
- Row count thresholds
- Duplicate business keys
- Valid data types
- Valid date ranges
- Mapping success rate
- Output schema match

Key Rule *Structural problems should fail loudly. Business exceptions should be logged with reason codes.*

Exception Handling Standard

Use structured try/except logic around notebook execution. Standard behavior:

- Attempt pipeline step
- Log failure if error occurs
- Preserve error message
- Raise the error after logging
- Do not suppress failures silently

Pattern	Behavior
Bad	try process → error occurs → continue anyway
Good	try process → error occurs → log failure → raise error → pipeline fails visibly

Retry Behavior Standards

Retries should be used only for likely temporary failures.

Category	Examples
Retry allowed	Temporary SQL connection issues, temporary API failures, temporary file access issues, transient Fabric execution errors
Retry not allowed	Missing required columns, invalid schema, duplicate business keys, stale source data beyond threshold, broken transformation logic, failed business rule validation

- Retry transient failures up to 2 times
- Log each failed attempt
- Fail visibly after final retry

Readability Standards

Every production notebook should include:

- Clear section headers
- Short reusable functions
- Comments explaining non-obvious business rules
- Config block at the top
- Meaningful variable names

- Row count summaries
- No hidden row drops
- No hardcoded mappings
- No unexplained transformations

Final Rule

Every production notebook should produce:

- A governed table
- A pipeline run log entry
- Rejected record details where applicable
- Clear failure visibility if validation fails